

Интеграция judge0 в moodle

Итерация 3

Запланированные задачи

- 1) Переход на асинхронную модель проверки — уход от блокирующих HTTP-запросов, которые могут зависнуть при нагрузке
- 2) Добавить файл `db/upgrade.php` — миграции БД для обновления плагина на существующих инстансах Moodle без сброса данных
- 3) Батчевые запросы к Judge0 — отправлять все тест-кейсы одним batch-запросом (`/submissions/batch`), а не по одному, для ускорения проверки
- 4) Портирование реальных задач с курса алгоритмов из CodeRunner в формат Judge0 — демонстрация обратной совместимости
- 5) Подсветка синтаксиса — интеграция Ace Editor или CodeMirror для подсветки кода в зависимости от выбранного языка
- 6) Горизонтальное масштабирование Judge0 — конфигурация нескольких worker-нод, балансировка нагрузки, тестирование под многопользовательской нагрузкой, измерить на локальной машине, графики
- 7) Обновление и расширение тестов — покрытие новых фич в `test_plugin_acceptance.py`

[Back](#)**Question 1**

Correct

Mark 1.00 out of 1.00

 [Flag question](#) [Edit question](#)[v2 \(latest\)](#)

SDFSDF

```
1 int solve(int a, int b) {  
2     return a + b;  
3 }  
4
```

**Поздравляем!**

Ваше решение успешно прошло все тесты.

Результаты тестирования:

#	Ввод (stdin)	Ожидалось	Ваш вывод	Статус
1	2 3	5	5	Accepted

Programming Language (Default)

?

Python 3.8.1

▼

Allowed Languages

?

✓ Python 3.8.1

✓ C (GCC 9.2.0)

C++ (GCC 9.2.0)

✓ Java (OpenJDK 13.0.1)

JavaScript (Node.js 12.14.0)

✓ C# (Mono 6.6.0.161)

✓ Bash (5.0.0)

Ruby (2.7.0)

Rust (1.40.0)

Kotlin (1.3.70)

PHP (7.4.1)

Checker Code

?

Reference Solution

?

Reference Solution Language

?

C++ (GCC 9.2.0)

▼

Compiler Options

?

Back

Question 1

Tries
remaining: 1

Marked out of
1.00

Flag
question

Edit
question

v8 (latest)

asdfgs

🚩 Language:

✓ Python 3.8.1

C (GCC 9.2.0)

Java (OpenJDK 13.0.1)

C# (Mono 6.6.0.161)

Bash (5.0.0)

```
6
7  int main(void) {
8      int a, b;
9      if (scanf("%d %d", &a, &b) != 2) return 0;
10     printf("%d\n", solve(a, b));
11     return 0;
12 }
13
```

Check

Finish attempt ...

Question 1

Correct

Mark 1.00 out of 1.00

 Flag question Edit question

v8 (latest)

asdfgs

 Language:

C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int solve(int a, int b) {
4     return a + b;
5 }
6
7 int main(void) {
8     int a, b;
9     if (scanf("%d %d", &a, &b) != 2) return 0;
10    printf("%d\n", solve(a, b));
11    return 0;
12 }
13
```

**Поздравляем!**

Ваше решение успешно прошло все тесты.

Результаты тестирования:

#	Ввод (stdin)	Ожидалось	Ваш вывод	Статус
1	2 3	5	5	Accepted

 Run tests again

 README.md	added test
 checker.c	added test
 reference_solution.py	added test
 student_solution_bad.c	added test
 student_solution_ok.c	added test
 testcases.txt	added test

README.md

🔗 Демо: лабораторная2_вариант3

1. Создать вопрос типа Judge0 в Moodle
2. Заполнить поле названия и поле описания
3. Язык: `C (GCC 9.2.0)` — `language_id=50`
4. Язык эталонного решения (Reference Solution Language): `Python 3.8.1`
5. Вставить содержимое `checker.c` в поле Checker Code
6. Вставить содержимое `reference_solution.py` в Reference Solution
7. Добавить тест-кейсы из `testcases.txt`
8. Проверить: вставить `student_solution_ok.c` → Accepted
9. Проверить: вставить `student_solution_bad.c` → Runtime Error

Back

Question 1

Correct

Mark 1.00 out of 1.00

Flag question

Edit question

v2 (latest)

lb3

Language: C (GCC 9.2.0)

```
29     return c;  
30 }  
31  
32 struct ListNode *list_get(struct ListStruct *list, int index, int *index_in_block)  
33 {  
34     if (index < 0)  
35         return NULL;  
36  
37     struct ListNode *cur = list->head;  
38  
39     while (cur) {  
40  
41         if (index < cur->count) {  
42             *index_in_block = index;  
43             return cur;  
44         }  
45  
46         index -= cur->count;  
47         cur = cur->next;  
48     }  
49 }
```



Поздравляем!

Ваше решение успешно прошло все тесты.

Задачи на следующую итерацию

- 1) Настройка лимитов по задаче — `cpu_time_limit`, `memory_limit` на уровне каждого вопроса
- 2) AJAX-проверка кода — отправка без перезагрузки страницы для ускорения UX
- 3) Ограничение скорости кнопки «Проверить снова» — ожидание 5–10 сек
- 4) Разработка инструкций
- 5) Подготовка финальной презентации